

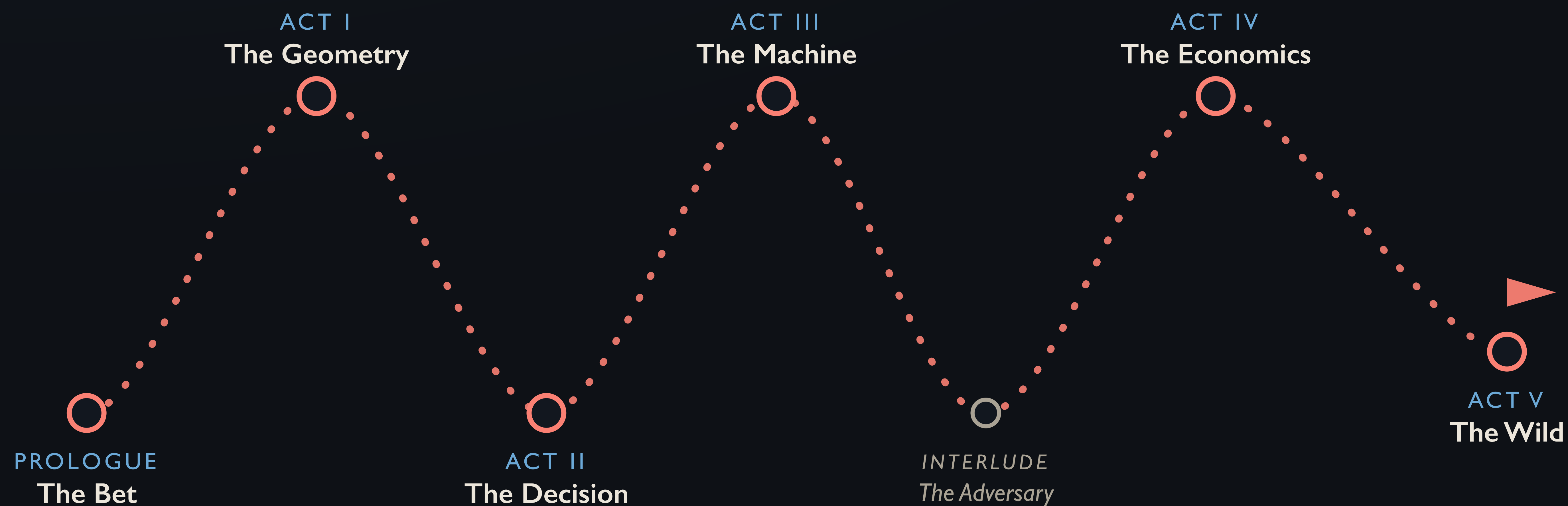
The Stones in the River

Semantic Caching for Enterprise RAG

No one steps in the same river twice — but
some questions are stones. A cache is the
wager that Heraclitus was only mostly right.

THE MAP OF THE DAY

Today's learning journey





PROLOGUE · THE JOURNEY
BEGINS

The Bet

what a semantic cache
actually is — and what it is
not

A cache is a bet that the past will repeat

Every cache ever built — CPU, CDN, memoize — bets on **exact** repetition.

The semantic cache raises the stakes: it bets that **meaning** repeats even when **wording** does not.

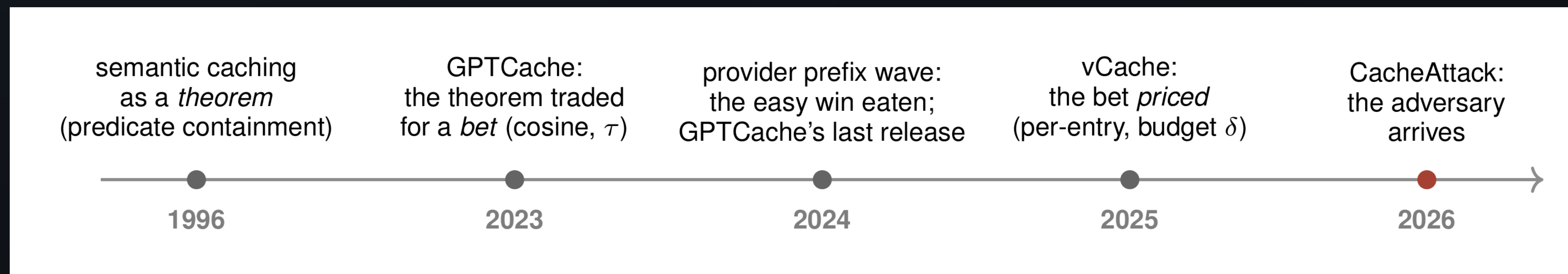
Same question, different clothes → serve the stored answer, skip the pipeline.

"There are only two hard things in computer science: cache invalidation and naming things."

— Phil Karlton

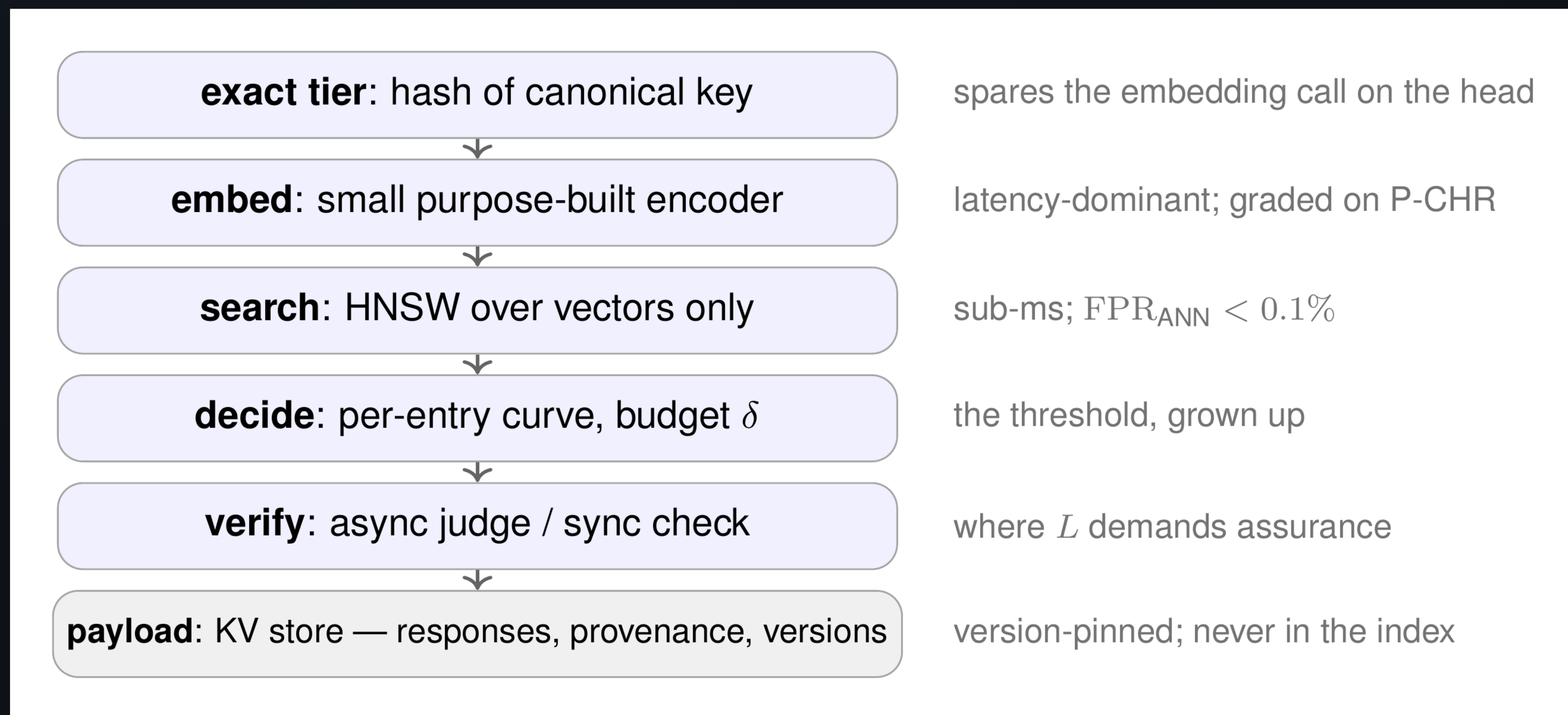
The semantic cache manages to be both.

From miracle to specialist instrument



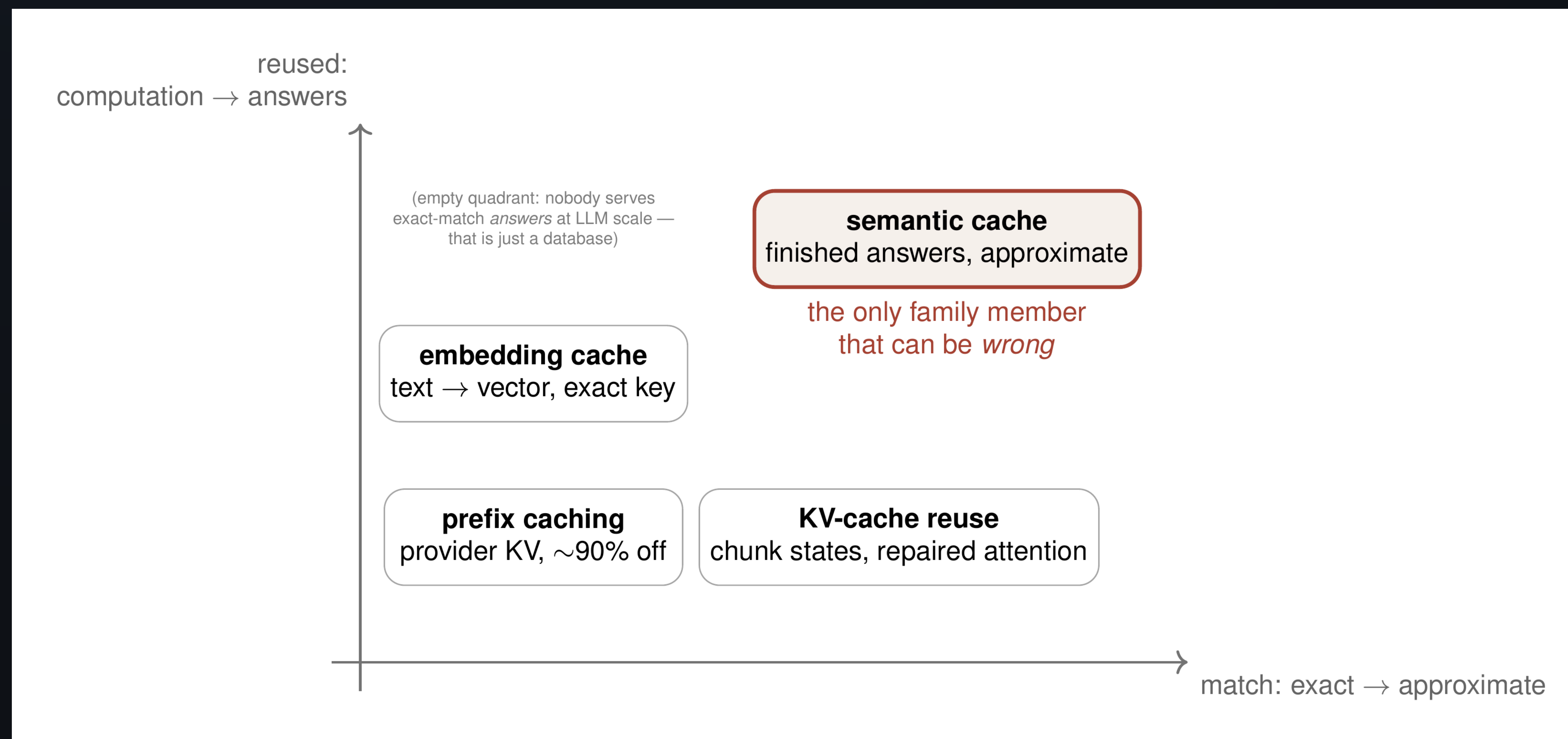
*GPTCache (2023) founded the field and is now abandoned — survived by its own citation. It did not die of a rival cache. It died of a **price change**. Act IV explains.*

One lookup, three doors



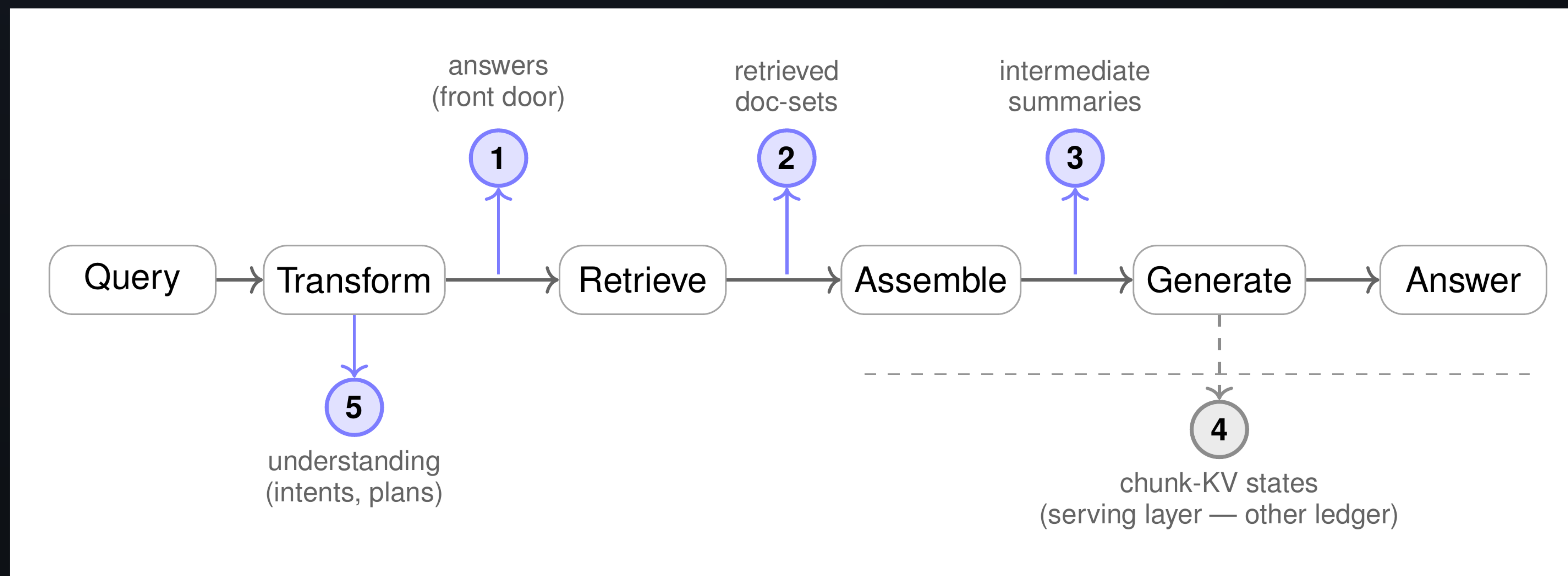
*Exact-hash tier first (microseconds, free), semantic tier second (evidence, not proof), full pipeline last — and every generation **writes back**. This is the Front Door pattern.*

What it is not



One sentence per border: not prefix caching (exact tokens, provider-side), not retrieval (which finds, never answers), not memory (which is about you, not about the question).

Five layers where a cache can live



*Embedding, retrieval, rerank, **answer**, and agent-state. Everything in the headlines is layer four — and layer four is where all five dangers concentrate.*

Answers are not KV blocks

- **Answer altitude** — cache **what was said**: semantic keys, approximate hits, correctness risk
- **KV altitude** — cache **computation**: exact prefixes, zero risk, the provider's business

The ledgers never mix. Every confused architecture we will meet today mixed them.



MILESTONE · ACT I OF V

The Geometry

why a hit is never an
accident — concentration of
measure on the hypersphere

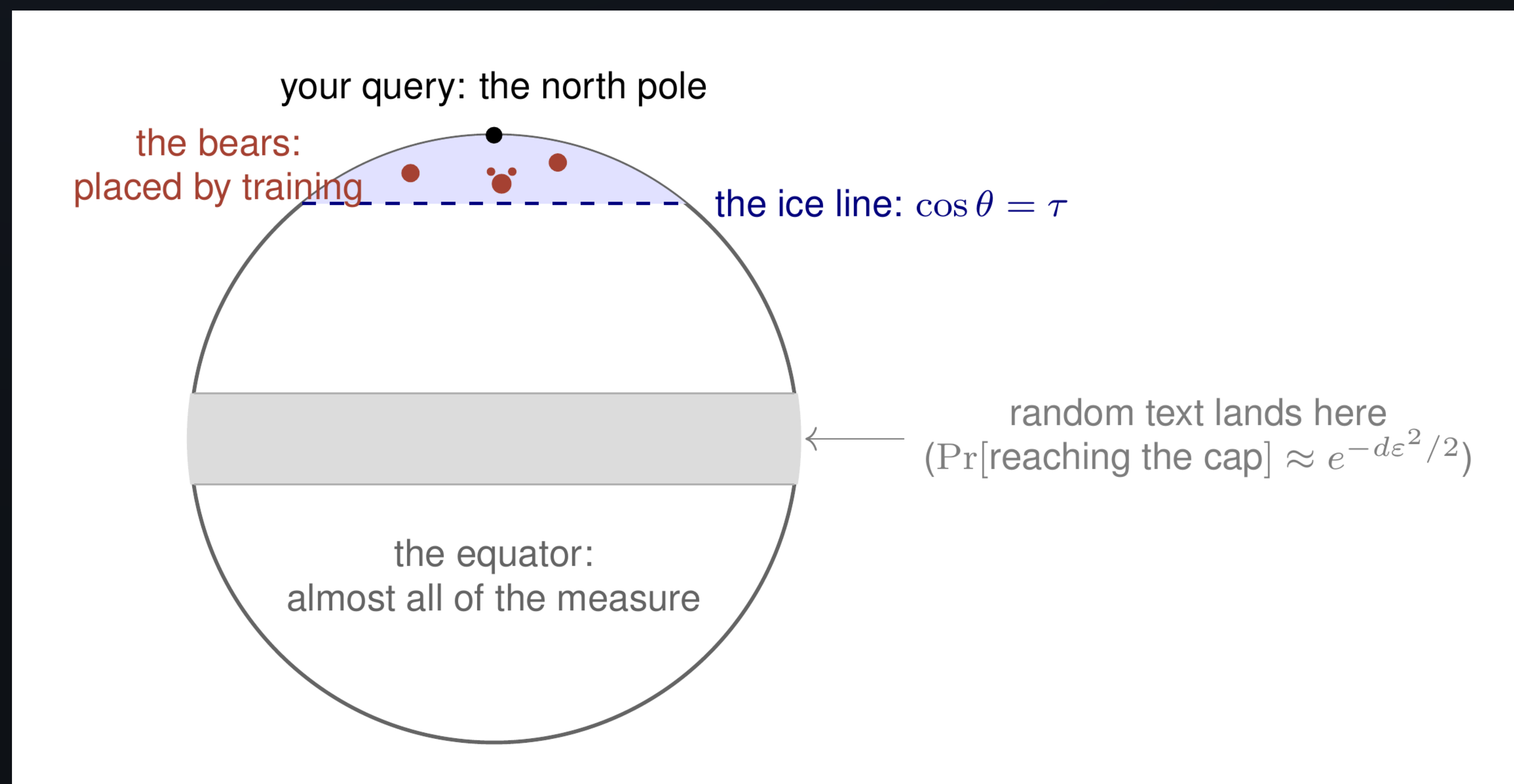
Life on the hypersphere

Embeddings live on the unit sphere in $d \approx 768$ dimensions. Intuition from $d = 3$ is not merely weak up there — it is **wrong with confidence**.

$$\sigma(\{x : \langle x, p \rangle \geq \varepsilon\}) \leq e^{-d\varepsilon^2/2}$$

*Read it aloud: the fraction of the sphere within a cap around any pole **dies exponentially in d** . Almost everything lives at the equator.*

Polar bears on the polar cap



Your query is the north pole; the cap above the ice line $\cos \theta = \tau$ is its habitat. Random text **cannot** reach it: at $d = 768$, $\varepsilon = 0.2$, chance mass $\approx e^{-15.4} \approx 2 \times 10^{-7}$.

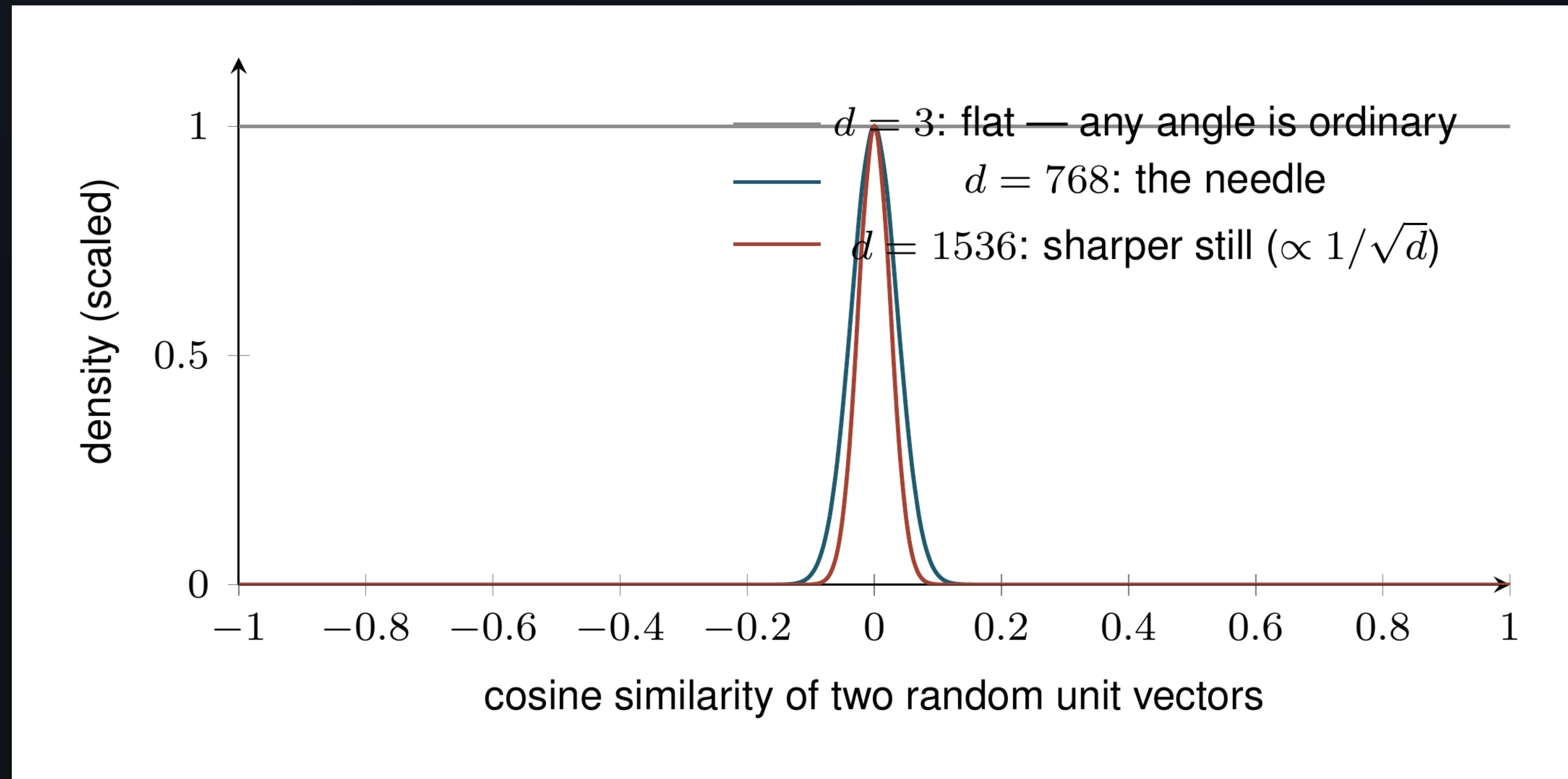
ACT I · THE DOCTRINE

Nothing wanders into the cap

The equator holds all the measure, so chance neighbors are annihilated. Whatever you find inside the ice line was **placed** there — by training, by shared meaning, or (hold the thought for the Interlude) by an adversary.

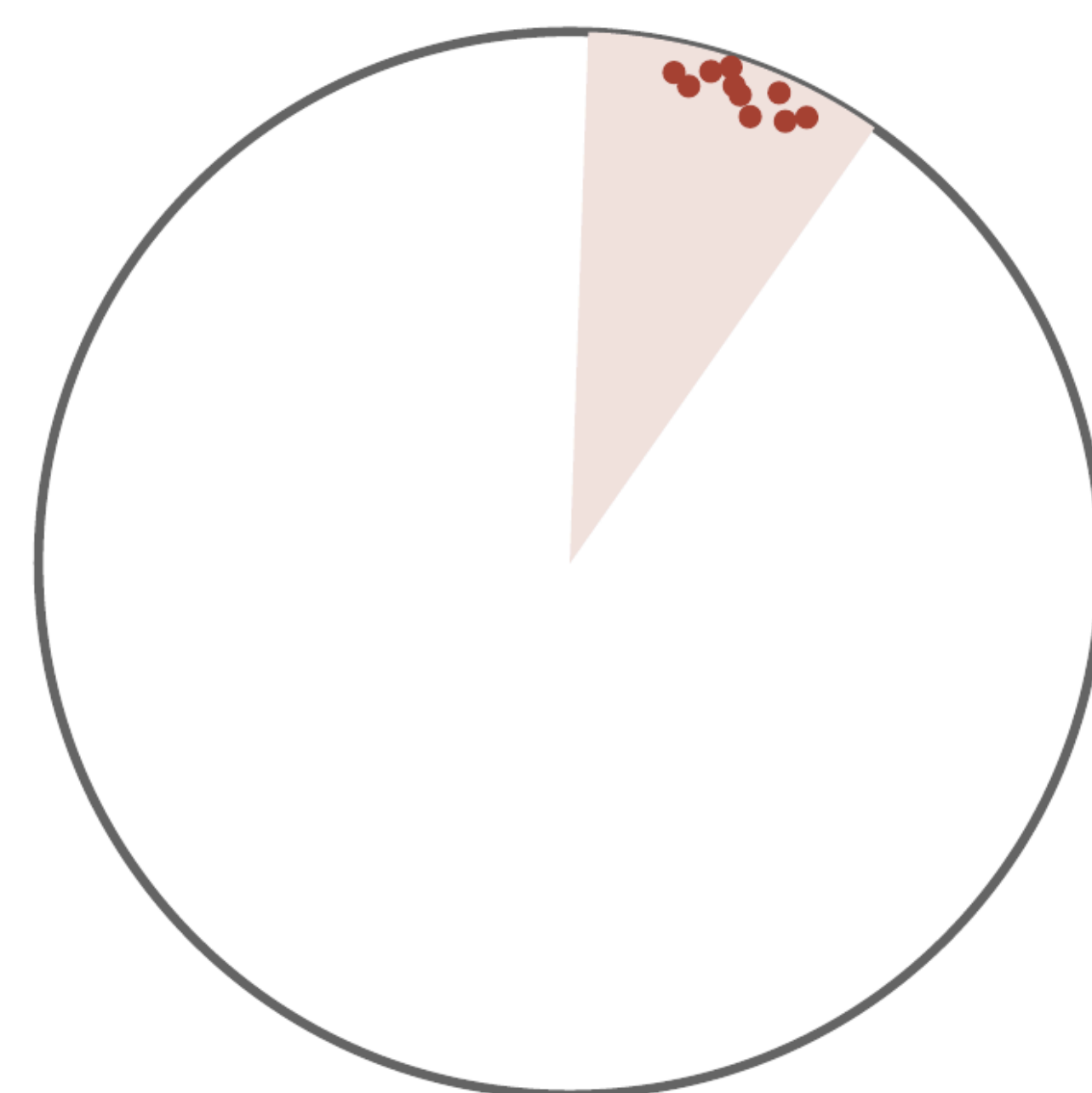
Every hit is a caused event. The bears are always somebody's doing.

Real encoders squeeze the sphere

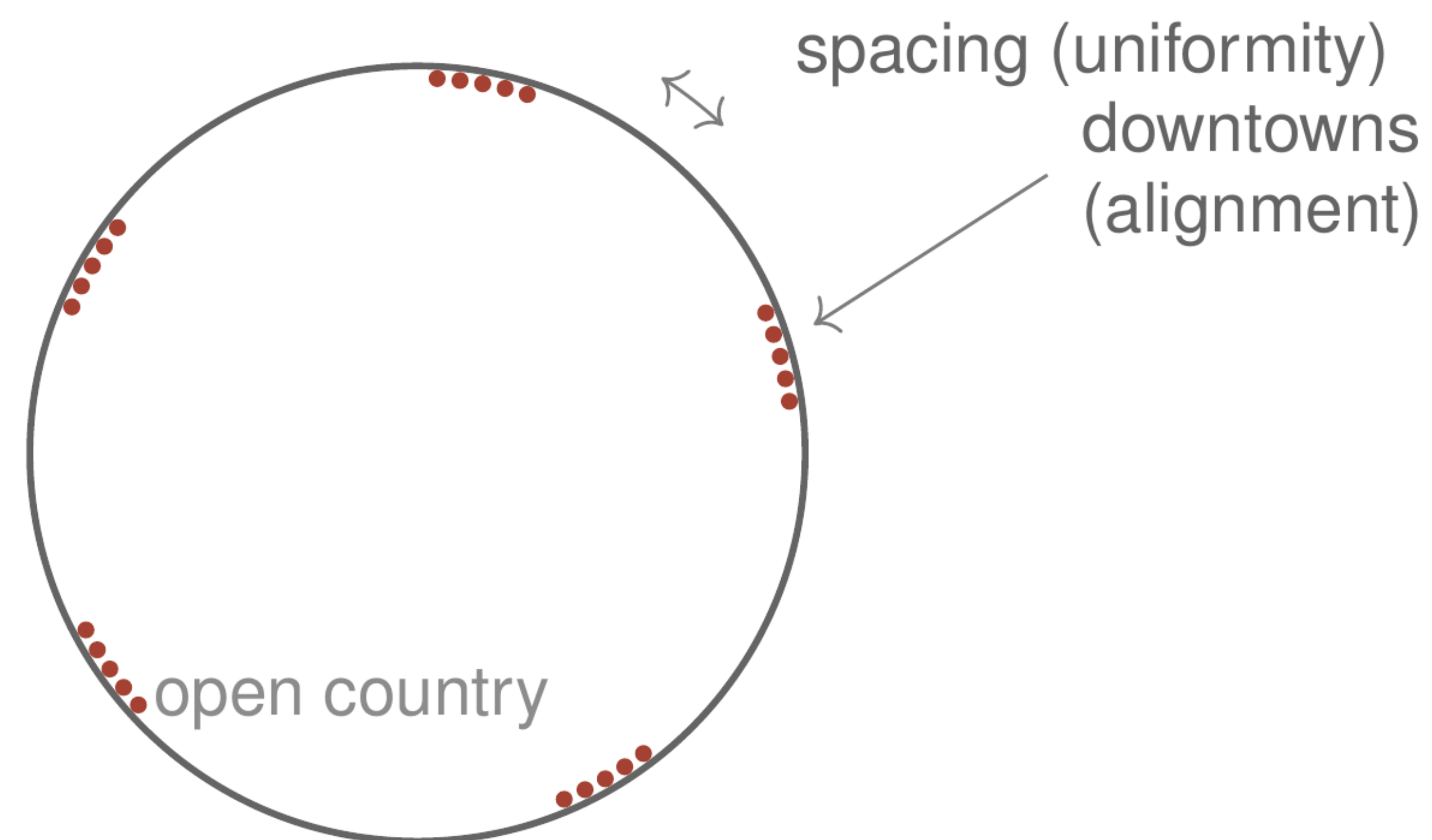


*Anisotropy: off-the-shelf embedders use a narrow cone — everything is cosine-similar to everything. The whole domain is Fremont. Raw cosine numbers are **encoder-dependent**, never portable.*

The metropolis effect



before: the whole domain is Fremont (anisotropic cone)

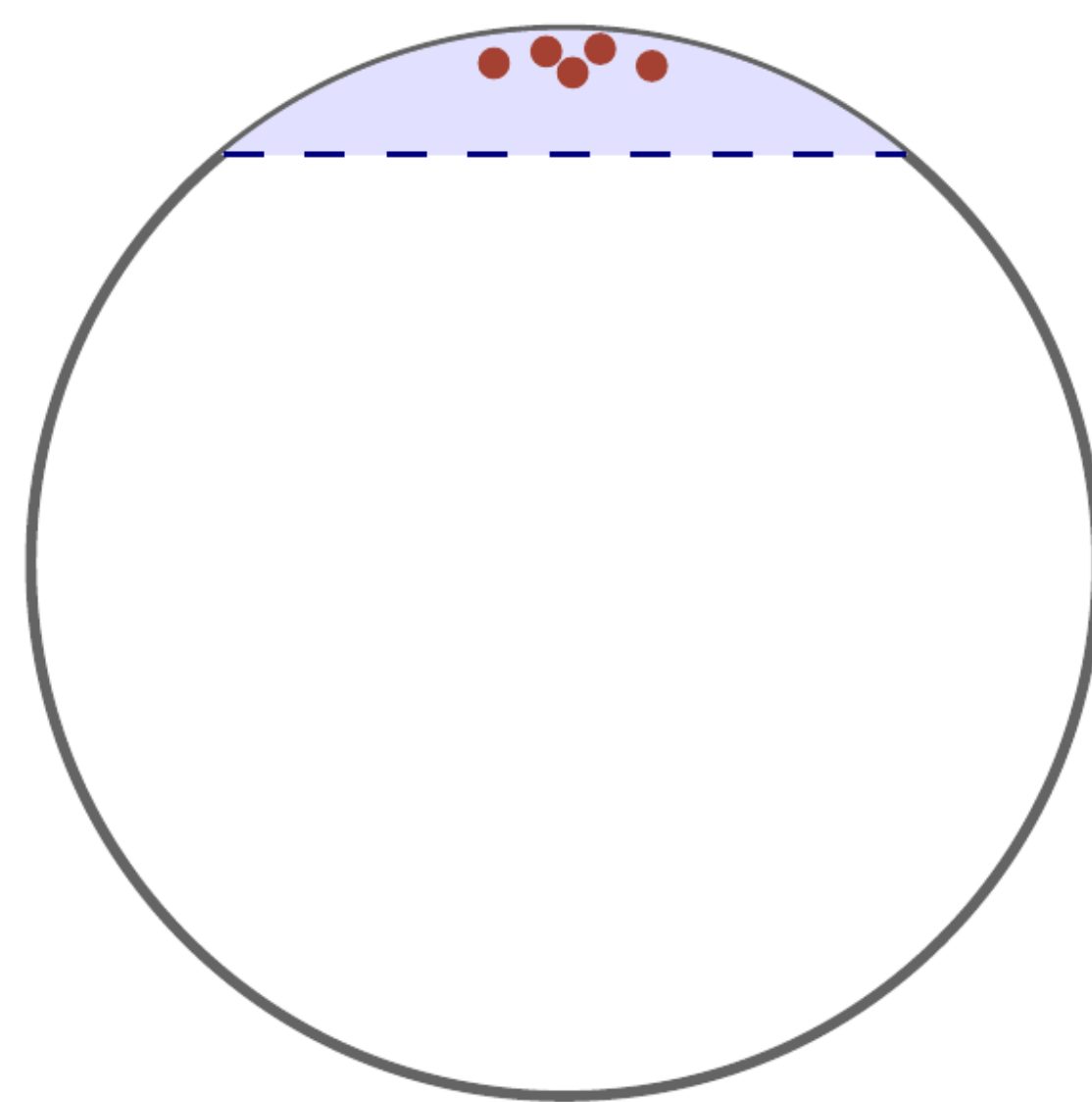


after: the urbanized sphere (the metropolis effect)

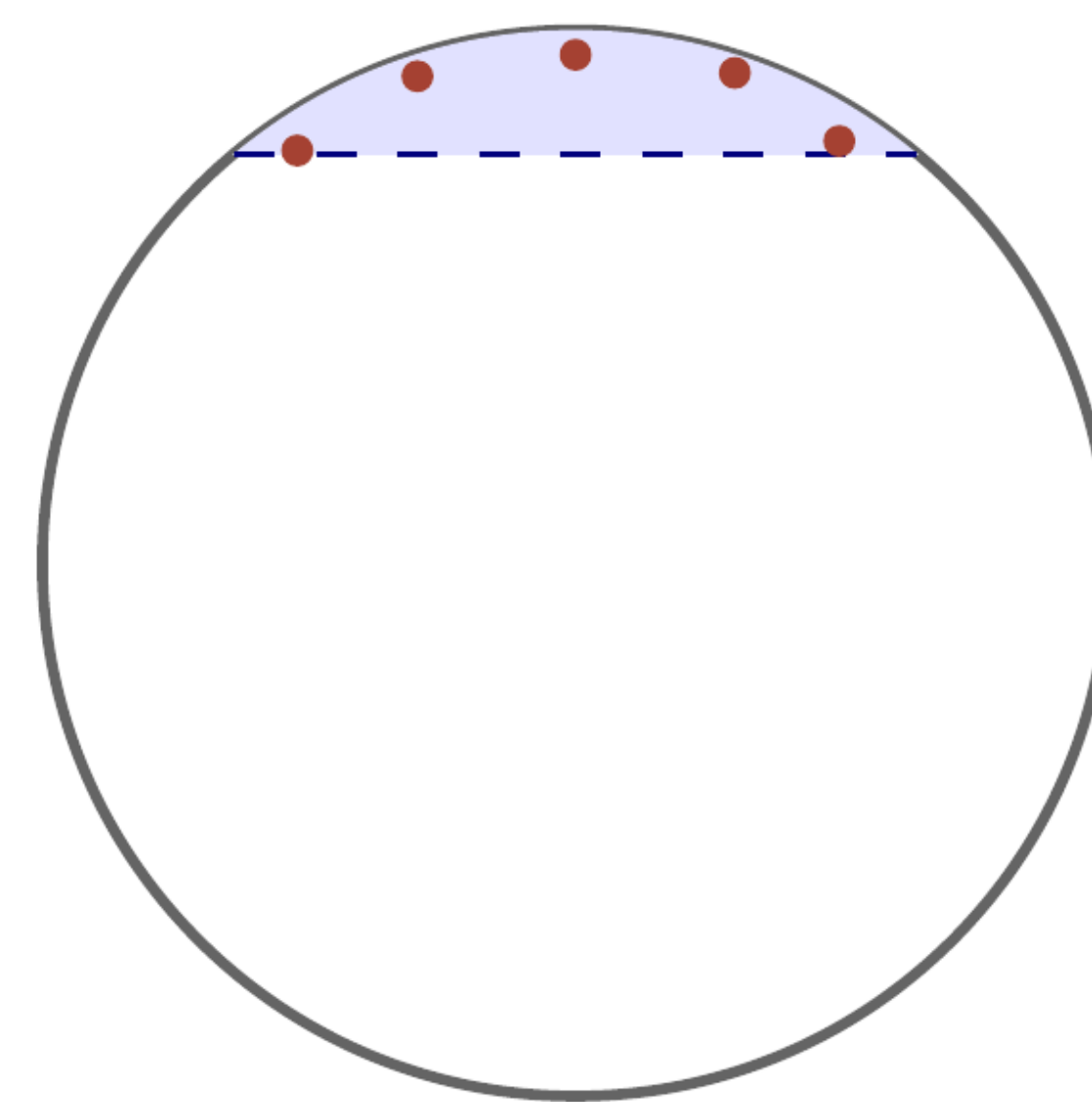
*Domain fine-tuning spreads the cone across the sphere — and condenses tight downtowns of near-synonyms. More surface used, **sharper** local clusters. Both matter to a cache.*

Temperature is the Boltzmann factor

same threshold, different encoder temperature:
 τ means a different thing on each sphere



low σ : tight downtowns —
the same τ captures the whole cluster

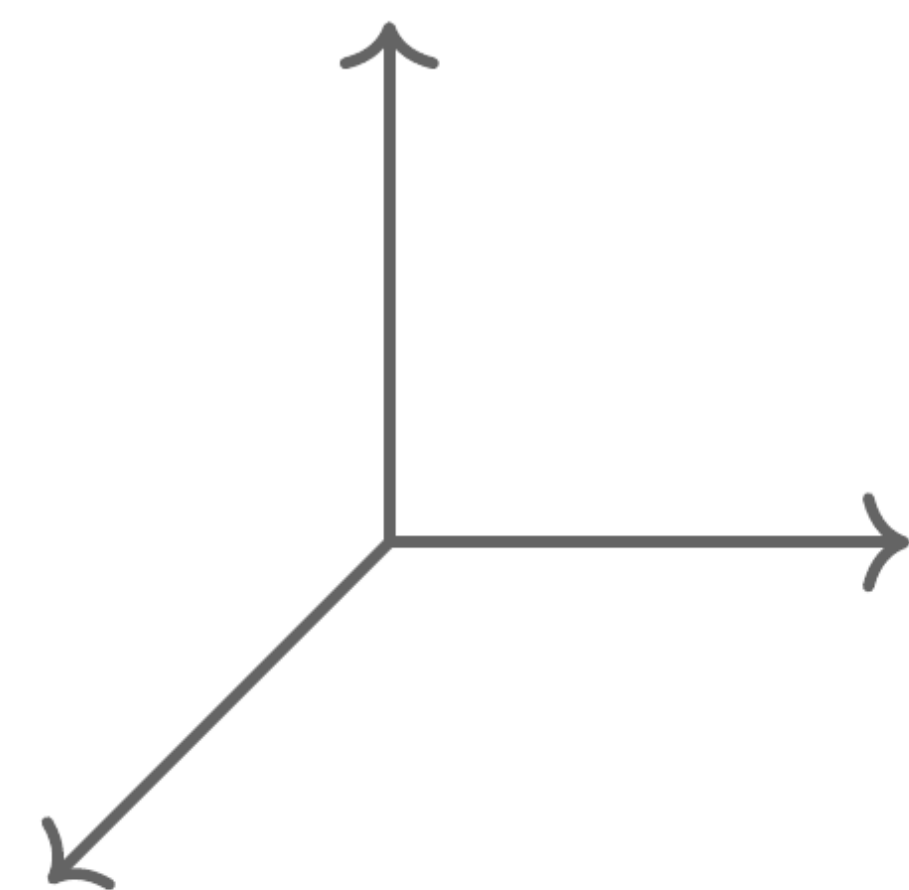


high σ : semantic sprawl —
the same τ slices the cluster in half

*Contrastive training optimizes alignment vs uniformity (Wang–Isola); the temperature $1/\beta$ sets how hard neighbors repel — **statistical mechanics on the sphere**, downtowns as condensation. Lower T , tighter metropoli.*

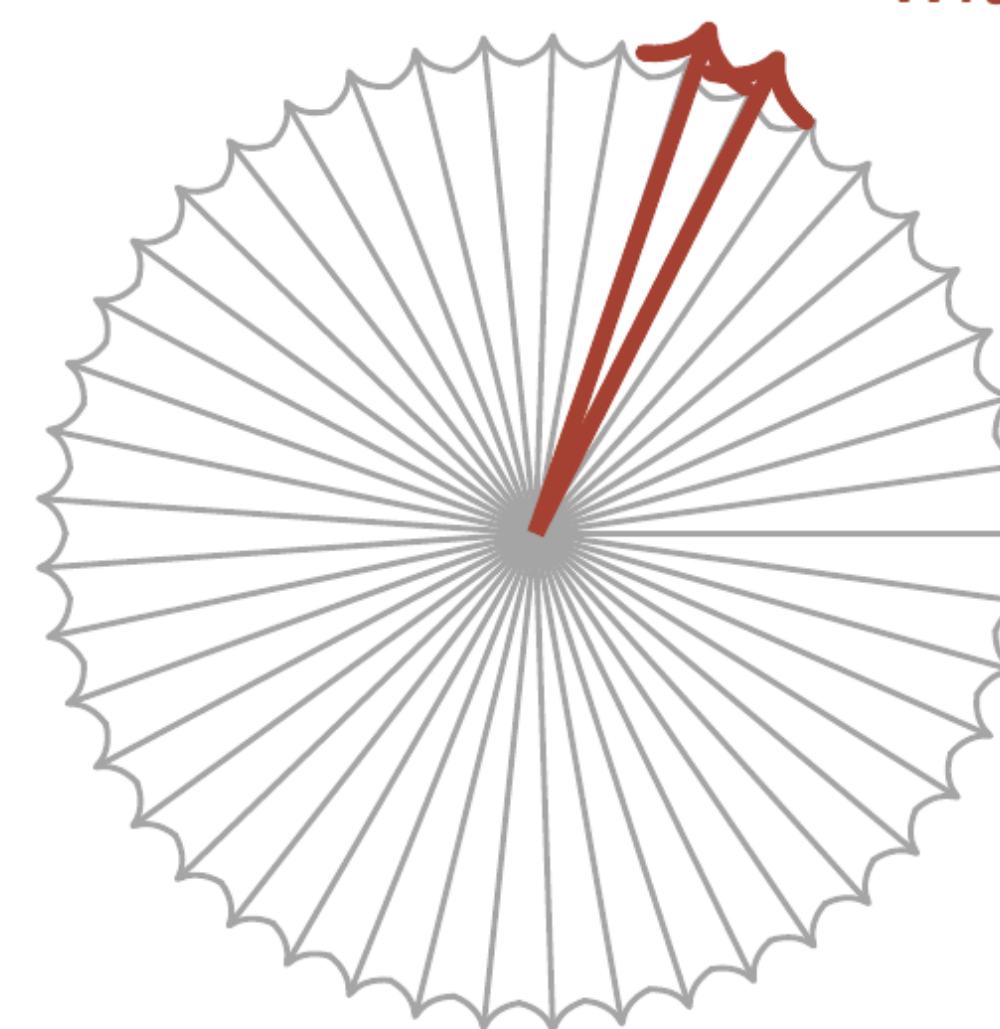
Exponential room at the top

strict orthogonality:
 d addresses



... one per dimension

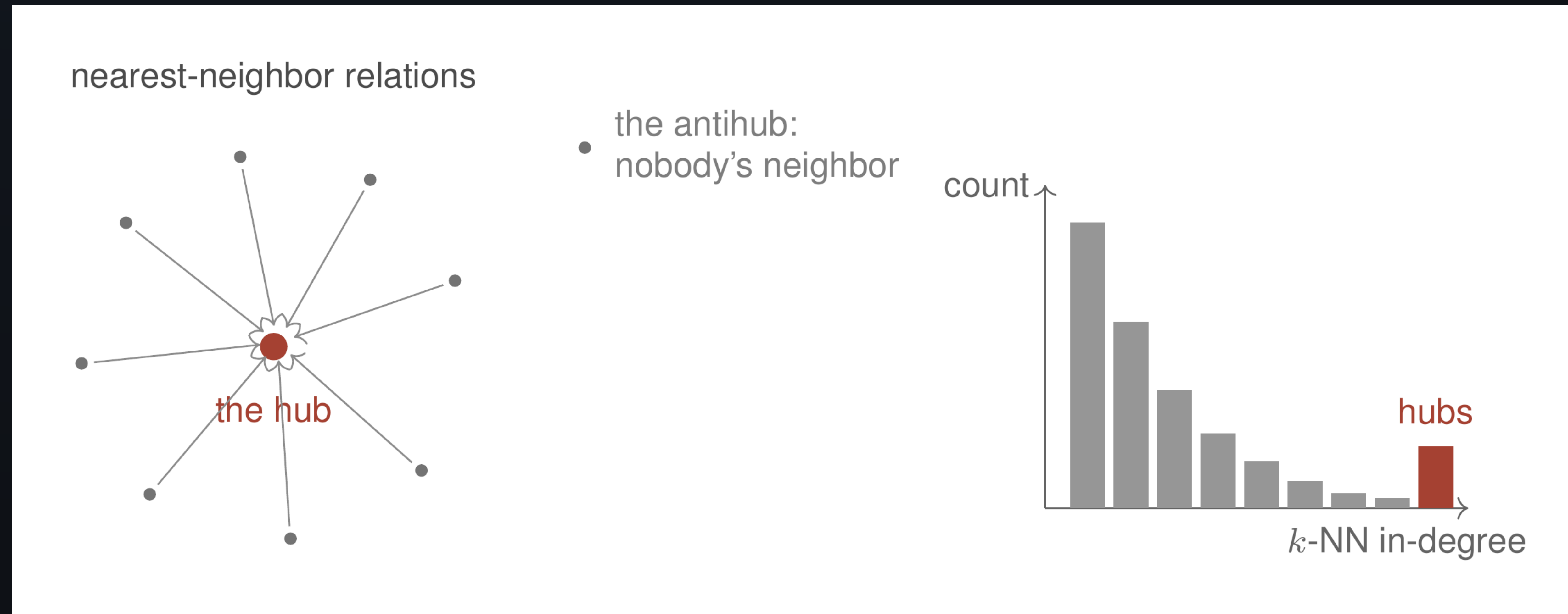
ε -near-orthogonal:
 $e^{c\varepsilon^2 d}$ addresses



Basel III and Basel IV:
close, yet distinct addresses —
with empty ones between

*Quasi-orthogonal capacity: the sphere holds $e^{c\varepsilon^2 d}$ nearly-orthogonal directions. Near-identical meanings need not sit on top of each other — **high dimension is the cache's friend**, not its enemy.*

Hubs: the town squares of the sphere



*In high-d nearest-neighbor graphs, a few points become everyone's neighbor. A hub in your cache is a **serial wrong-answer machine**. Diagnosis: k -occurrence skew; treatment: CSLS.*

Quiz I — the bear census

A colleague claims your cache's hits "could just be coincidence — the index returned something nearby."
Sphere: $d = 768$, cap $\varepsilon = 0.2$, and you probe with **ten million** random queries.

Compute the expected number of random probes that land in one cap.

*Then **say it back** in one sentence: what does finding a point in the cap prove?*

Quiz I — answered

$$e^{-d\varepsilon^2/2} = e^{-768 \cdot 0.04/2} = e^{-15.36} \approx 2 \times 10^{-7}$$

Ten million probes $\times 2 \times 10^{-7} \approx$ **two** arrivals — from **random** text. Real queries are not random, which is exactly the point.

*Say-back: chance is annihilated, so a point in the cap was **placed** there — every hit has a cause, and the cache's job is to decide **which** cause.*



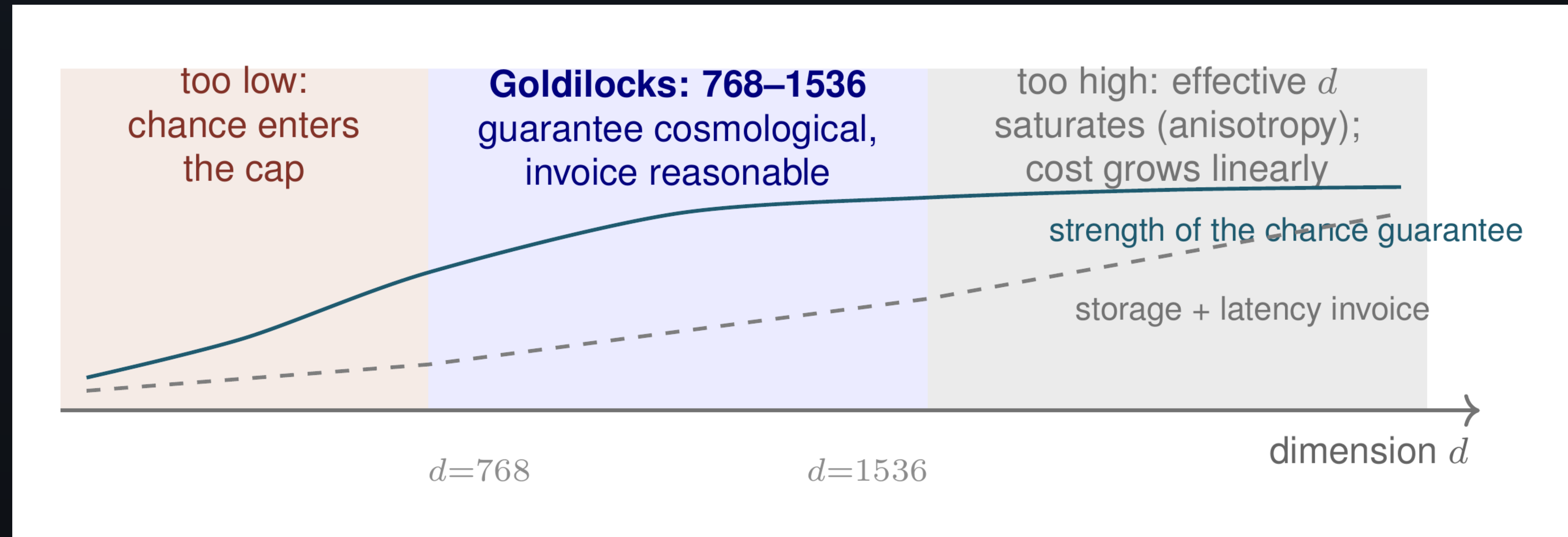
II

MILESTONE · ACT II OF V

The Decision

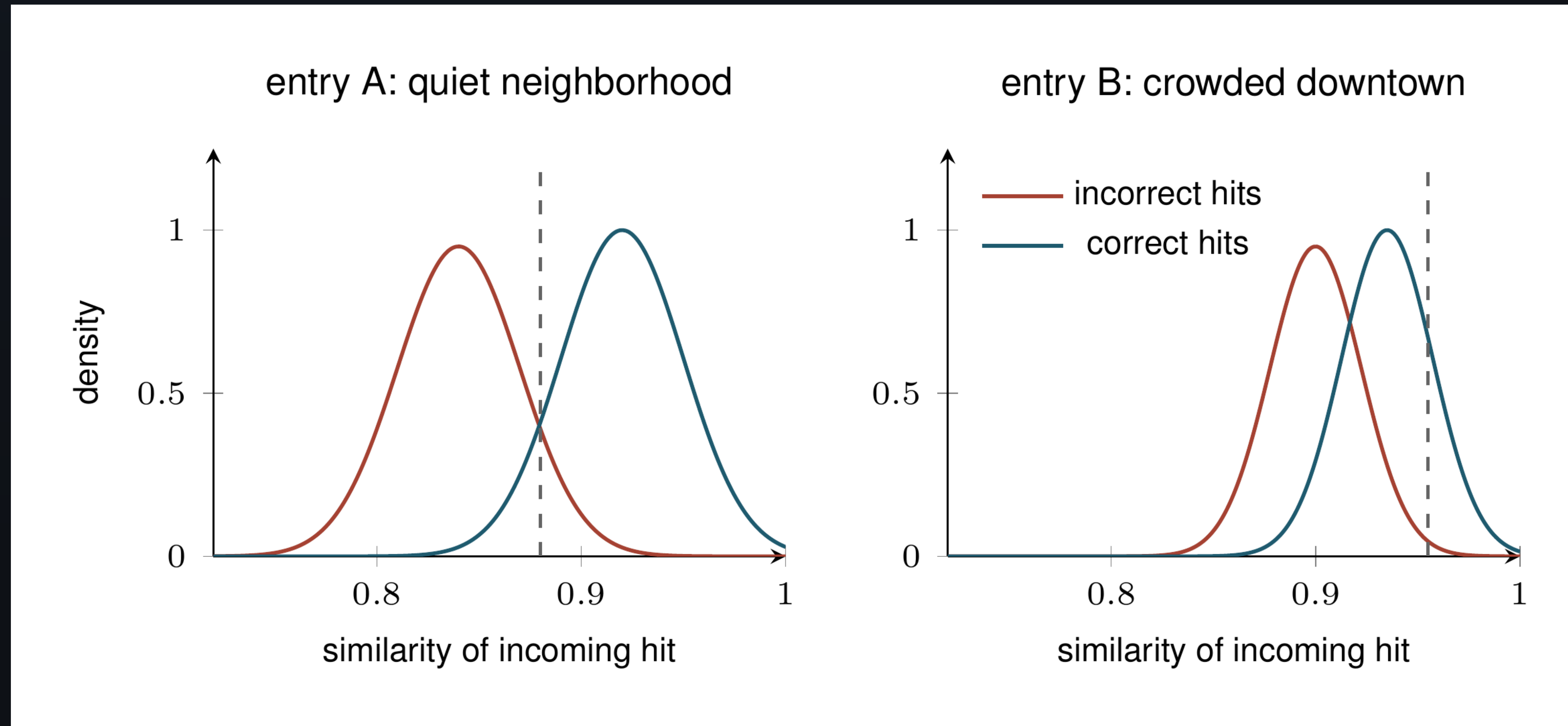
from a threshold you chose
to an error you can afford

The Goldilocks band



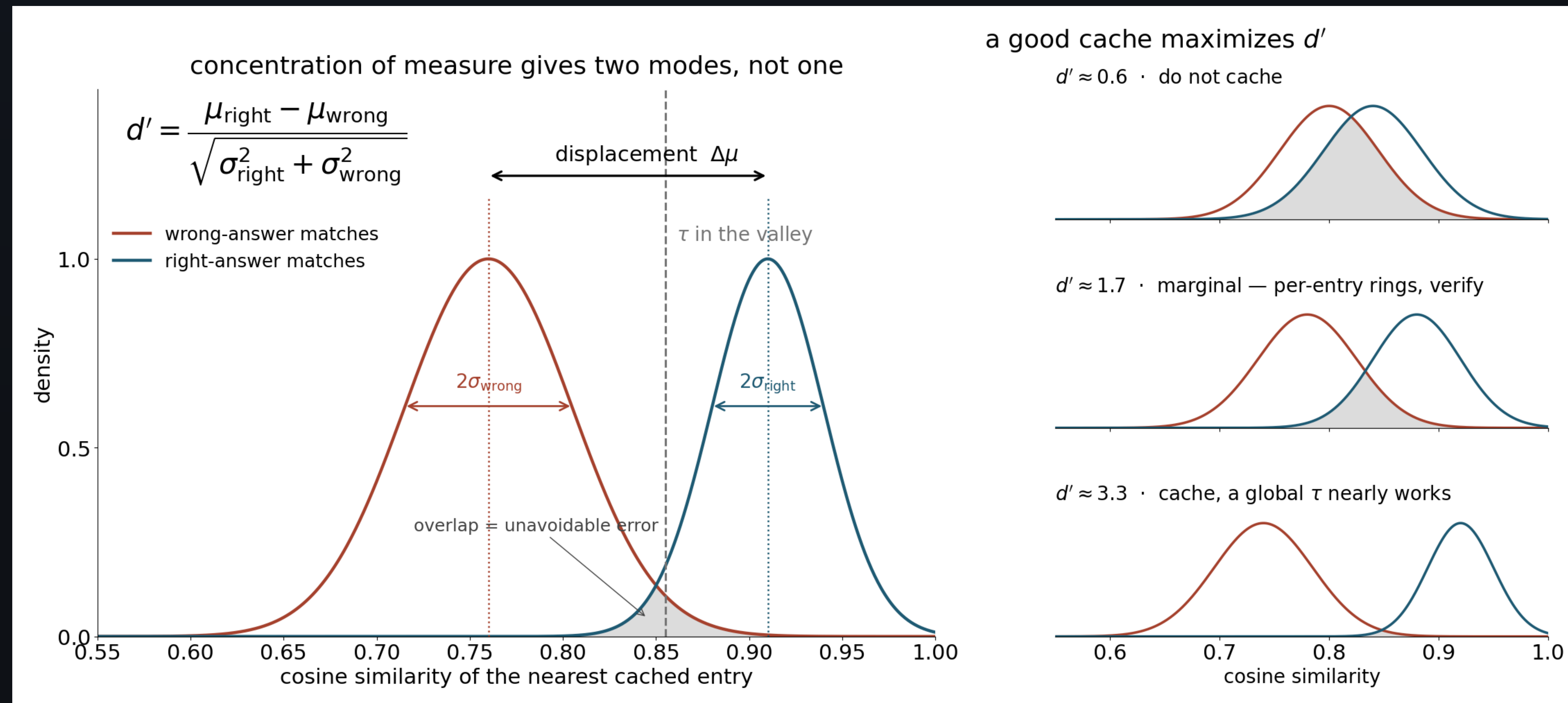
*Too strict: the cache starves, paraphrases miss. Too loose: wrong answers served with confidence. Every team starts by hunting the "right" τ — the literature's verdict: **there isn't one**.*

Within the cap, distributions overlap



*Concentration proves outsiders can't get in. It says **nothing** about insiders: per entry, correct and incorrect matches overlap in cosine. No global scalar can cut both cleanly. The theorem hands off to policy.*

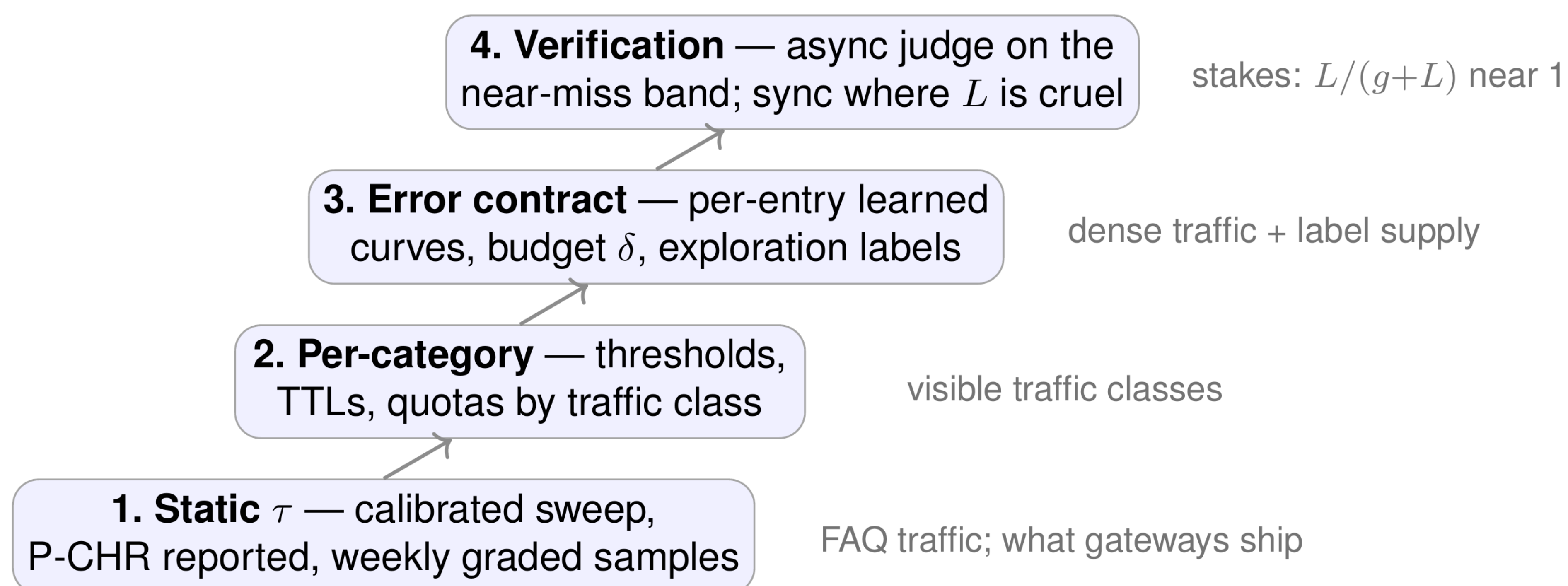
"Engineers often do efficiently what shouldn't be done at all"



Concentration gives **two** tight modes, not one spike. Cacheability is their **displacement** scaled by width, d' : a good E maximizes it, τ sits in the valley, the grey overlap no τ

Four rungs from dial to contract

choose your rung by $L/(g + L)$;
whatever the rung, state your demonstrated error



Products ship rung one; the literature lives on rungs three and four — that two-rung gap is the most exploitable fact in this deck.

ACT II · THE CONTRACT

vCache: promise δ , not τ

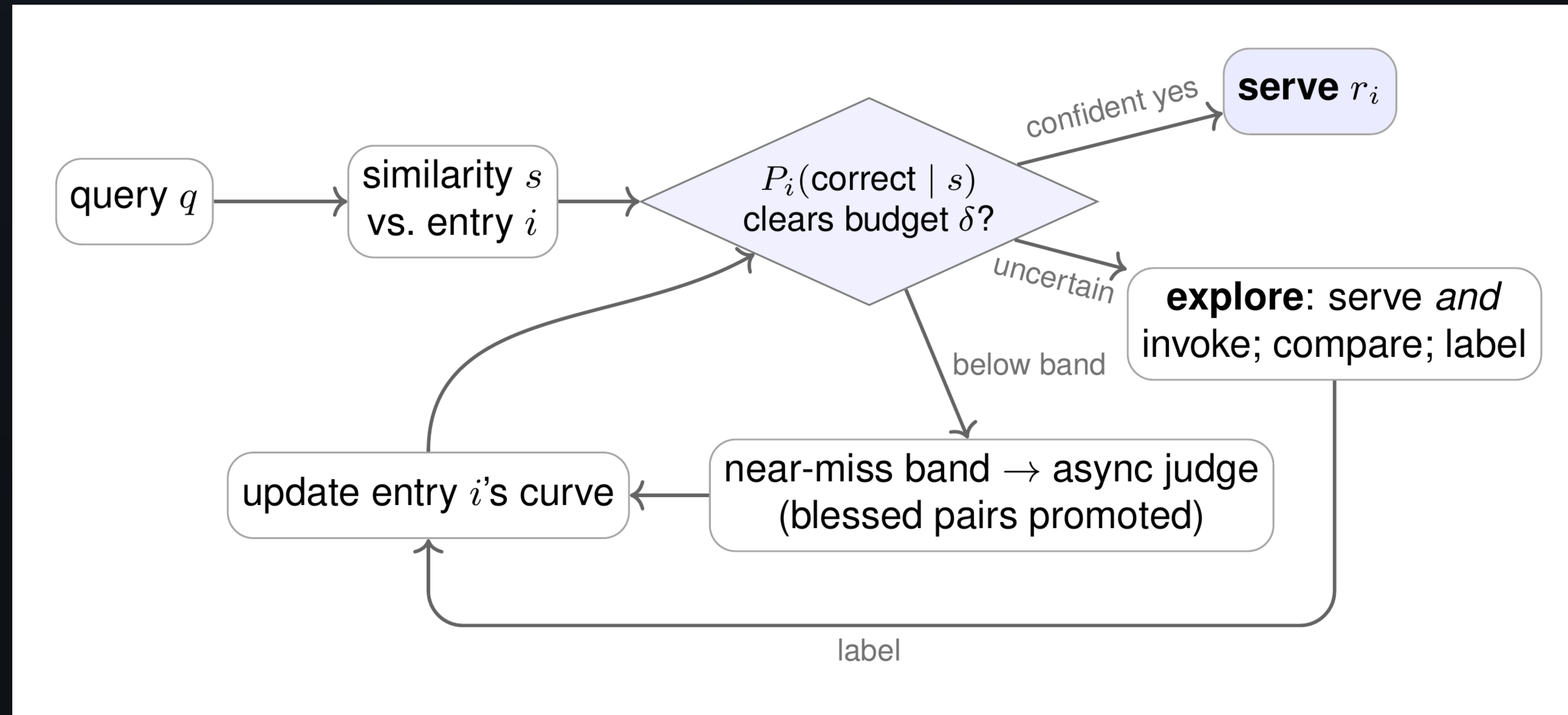
Replace "similarity ≥ 0.92 " with a **stated error budget**: serve only when

$$\Pr[\text{wrong serve}] \leq \delta$$

learned per entry from exploration labels — and report the **demonstrated** error, weekly, next to the hit rate.

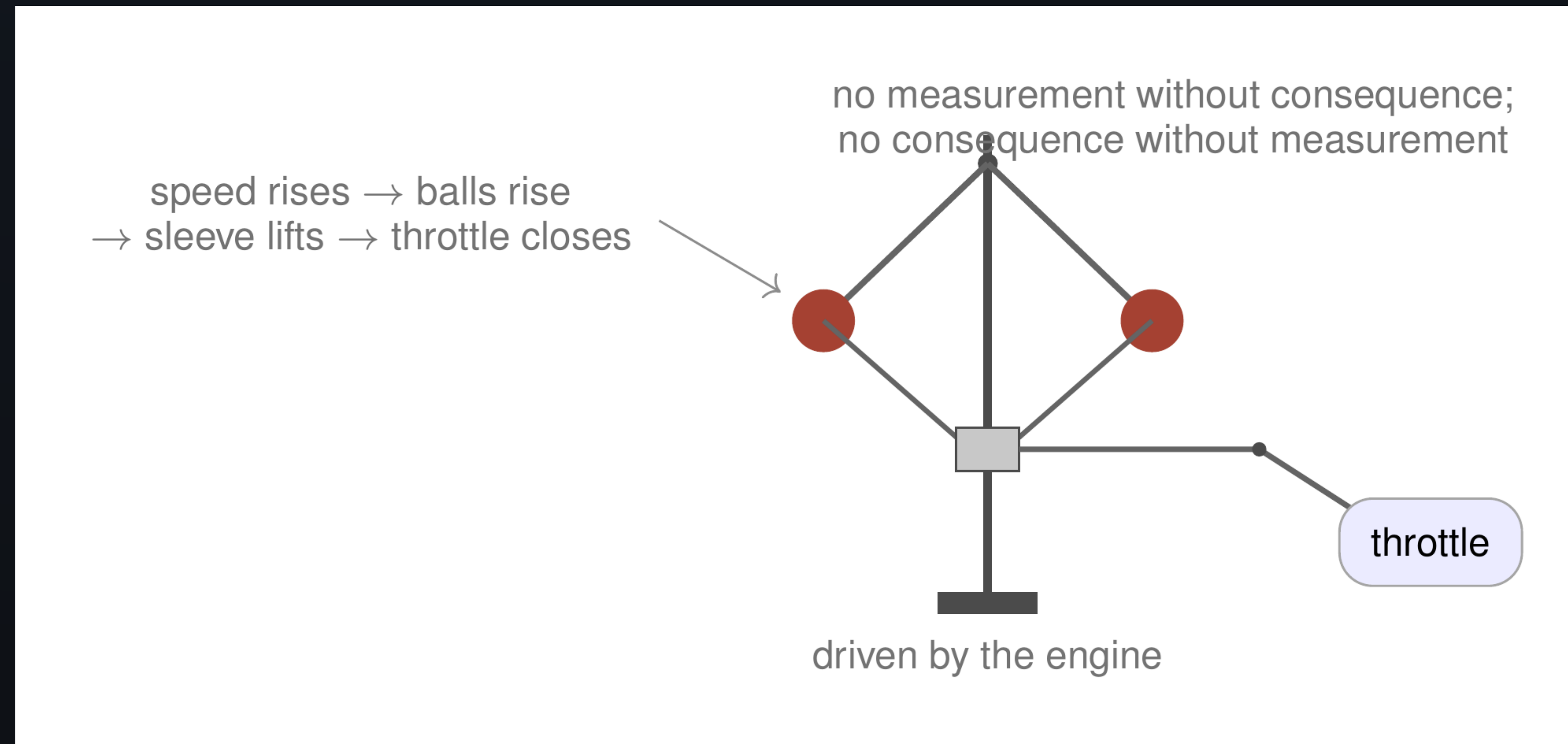
*A threshold is a hope. A contract is a **measurable promise** — and it survives embedder swaps, because δ is portable and τ never was.*

The verified path



Similarity is evidence; a judge converts evidence into assurance — asynchronously blessing the near-miss band, synchronously where wrongness is unaffordable. Verification is the toll booth between probably and served.

A governor, not a gate



*Maxwell, 1868: the centrifugal governor holds engine speed by feedback, not by a fixed valve. The error contract is the same machine — exploration measures, the policy corrects, δ is the set-point. **Fourth appearance of this architecture in the course.***

ACT II · THE ECONOMICS OF ONE SERVE

Value of a Serve

$$\text{VoS} = pg - (1 - p)L - \kappa \quad \Rightarrow \quad \text{serve iff } p > \frac{L + \kappa}{g + L}$$

One law, three doors: **VoL** priced a lookup, **VoR** priced a recall, **VoS** prices a serve. The cache's signature term is $(1 - p)L$ — the stale answer served with confidence.

Stakes set the bar: when being wrong is twenty times worse than being right is good, you need $p > 0.95$ before touching the dial.

Quiz 2 — one threshold to rule them all

Two entries share your cache. **A**:

"What is RAG?" — paraphrases match correctly down to cosine 0.83. **B**:

"What is the maximum daily dosage?"
— **wrong** matches score up to 0.96.

Pick the global threshold that serves both well:

(a) $\tau = 0.90$ **(b)** $\tau = 0.97$ **(c)**
 $\tau = 0.93$

Choose — then **say it back**: what does your choice do to the **other** entry?

Quiz 2 — answered: none of them

At 0.90: B serves poison. At 0.97: A starves — and B **still** overlaps. At 0.93: both failures at once. The options were a trap — **no scalar exists**; the fix is a per-entry policy: the error contract.

A student forced to pick from the options given is a model forced to answer from the context given.

*"None of these" **is** negative rejection — keep it loaded.*



III

MILESTONE · ACT III OF V

The Machine

the specialist embedder,
freshness against the world,
and caching for agents

Fine-tune the matcher, not a generalist

Cache matching is **paraphrase detection**, not topical similarity — "cancel my card" vs "cancel my flight" is a 0.9-cosine catastrophe. So: a small encoder (ModernBERT-class), fine-tuned with **curriculum learning**, a **decoupled loss**, and **hard negative mining**.

The FT-course trinity returns: easy pairs first, separate the pull-together from the push-apart, and feed the encoder the near-misses it almost believed.

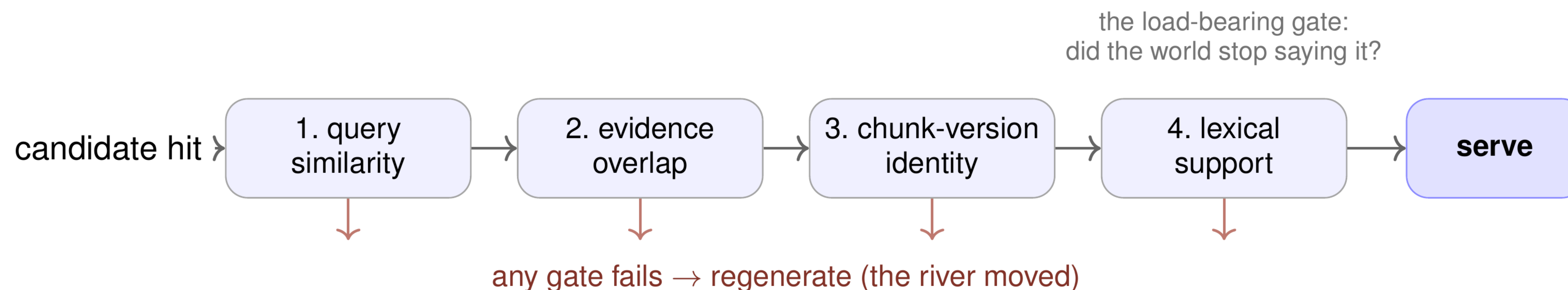
ACT III · THE SECOND SALARY

The near-miss mine

The band just below τ concentrates exactly the pairs your encoder confuses — logged, clustered, they become **hard negatives** for the next fine-tune, emerging-topic alerts, and skill candidates.

*The cache is not only a saver; it is an **instrument** pointed at your own embedder. Mine the band — it is the cheapest training data you will ever get.*

The evidence gauntlet



*TTLs guess; the world doesn't consult them. GroundedCache checks the hit against **evidence**: similarity, evidence overlap, chunk-version identity, lexical support — serial gates at hit time, TTL demoted to backstop.*

ACT III · INVALIDATION

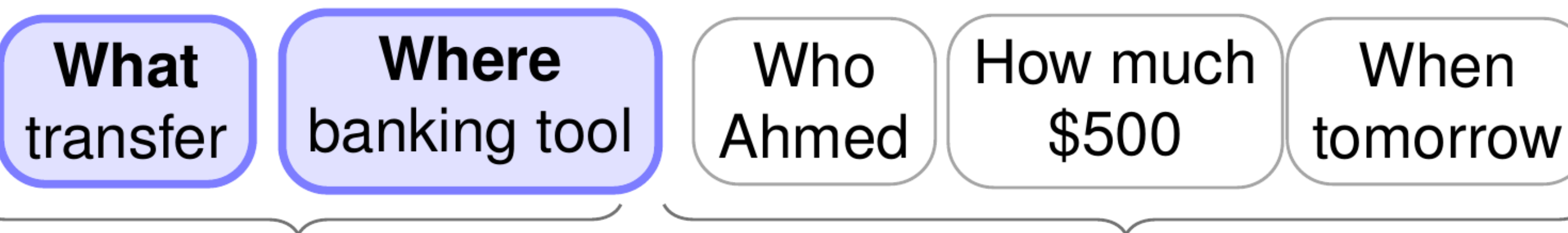
The version pin

An entry minted under one geometry must **die with it**: embedder version and index version live in the entry schema; mismatch is an unconditional miss; migration is re-embedding, not hope.

Silent embedder upgrades corrupt neighborhoods in ways that look like drift. Pin the version — invalidation is tied to index versions, full stop.

Canonical keys for agent traffic

“Transfer \$500 to Ahmed tomorrow morning”



the cache key: the route,
recognized instantly

never in the key: parameters,
re-bound live at execution

*Agent queries repeat in **shape**, not in wording. Canonicalize before keying — the W5H2 frame: what, where, who, when, why, how, how-much — parameters excluded and re-bound live.*

ACT III · AGENTS, CONTINUED

Cache the plan, not the answer

Agent state goes stale too fast to cache answers — but **structure repeats**: extract plan skeletons from successful episodes, adapt bindings live, promote persistent winners toward the skill library.

*At the agent altitude, the plan cache is the **only** safe answer-adjacent cache — and its graduation path ends in SkillCraft's promotion pipeline.*

Quiz 3 — pricing the serve

Voice support bot. A correct instant answer is worth $g = \$1$; a wrong one costs $L = \$20$ in cleanup; the lookup itself costs $\kappa = \$0.01$.

Compute the minimum win probability p^ that justifies serving.*

*Then **say it back**: why does the same cache need a different p^* for a refund-policy question than for a store-hours question?*

Quiz 3 — answered

$$p^* = \frac{L + \kappa}{g + L} = \frac{20.01}{21} \approx 0.953$$

Drop the stakes to $L = \$1$ (store hours): $p^* = 1.01/2 \approx 0.505$. Same law, different door.

*Say-back: the serving bar is set by the **asymmetry of consequences**, not by cosine — stakes move p^* , so one global dial can never price both questions.*

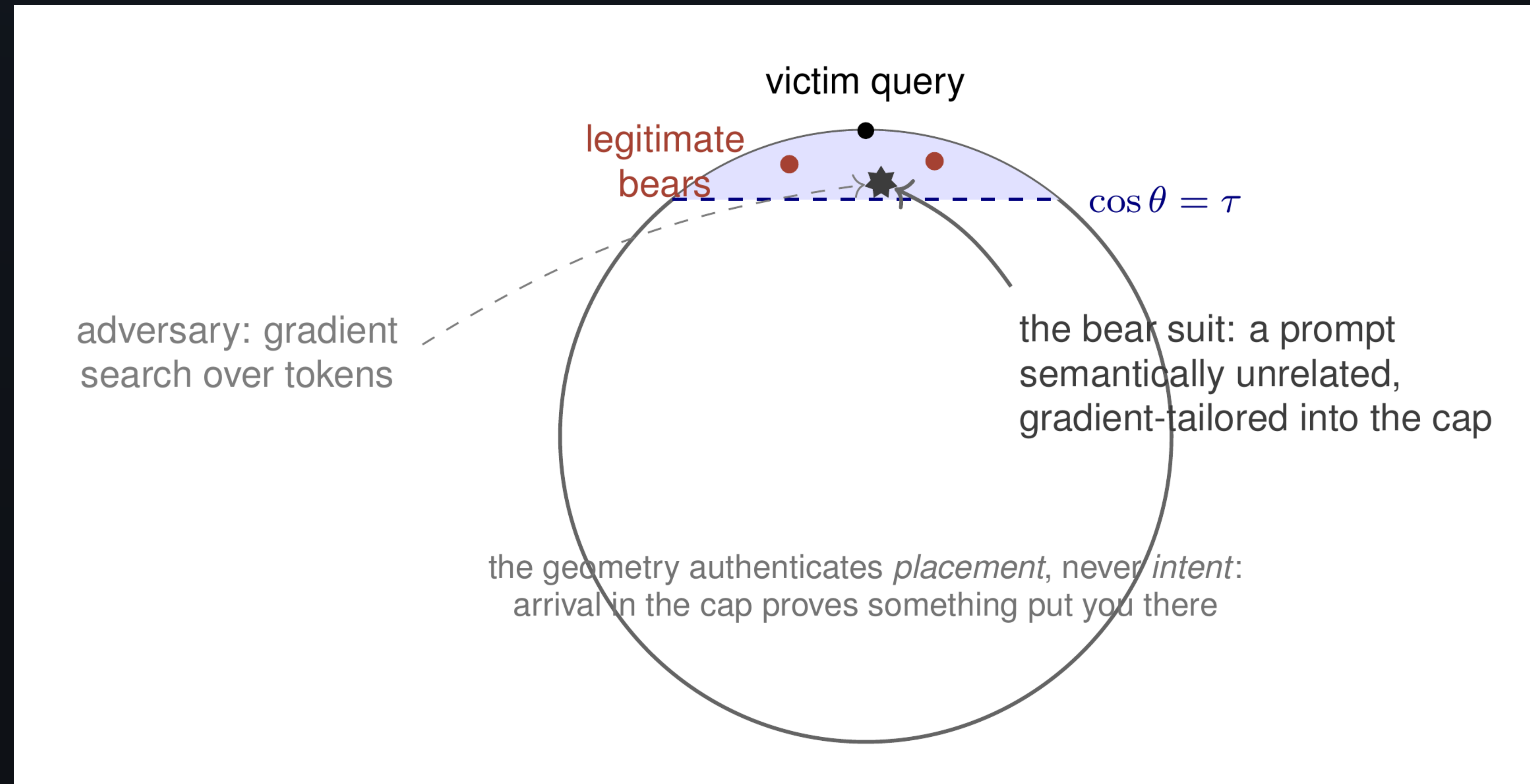


INTERLUDE · BETWEEN THE ACTS

The Adversary

locality is collision risk — the
theorem, weaponized

Placement can be purchased



*The geometry authenticates **placement**, never **intent**. Gradient search over tokens crafts a prompt that wears cosine above τ — arrival in the cap proves only that **something put you there**.*

INTERLUDE · THE THEOREM, INVERTED

Locality $\Leftrightarrow$ collision

The same concentration that makes hits **causal** makes crafted collisions **cheap to aim**: the attacker needs to reach one small cap, and the cap is exactly where your legitimate traffic lives. Adversarial hubness plants one entry that answers **many** queries.

Every property that makes the cache work is the attack surface. There is no configuration that is both open and safe.

THE STONES IN THE RIVER · SUPPORTVECTORS

Scope, verify — and pay for both

defense	what it buys	what it costs
key salting	raises attacker search cost	blurs honest neighborhoods
perplexity screen	rejects gibberish suits	ratchet, not a wall
centroid matching	no single-point nearest	coarser matching
verification	catches served nonsense	latency + judge cost
scoping	removes the target	hit rate (smaller populations)

*Per-tenant always, per-user where content personalizes — the scope key lives **inside the cache key**, not in an application check. Scoping halves shareable populations; verification taxes every guarded hit. Quote H **after** the defense, not before.*

Quiz 4 — the neighboring tenant

Two customers share your semantic cache. Tenant A's analyst asks "What was our Q3 revenue?" and the answer is cached. Tenant B's analyst asks the same words next morning.

Judge it: what must be true of the cache key for this to be safe?

Then *say it back*: why is an `if`
`tenant_id == ...` check in
application code not an acceptable
answer?

Quiz 4 — answered

The tenant ID must live **inside the key** — same words, different tenant, **structurally** different keyspace: the lookup cannot find the foreign entry, so there is nothing to leak.

*Say-back: a check forbids the leak; a scoped key makes it **impossible** — and the partition interlude of MemoryCraft said it first: structure beats vigilance, because vigilance has outages.*



IV

MILESTONE · ACT IV OF V

The Economics

after prefix caching ate the
easy win

ACT IV · THE KILLERS

Who killed the 2023 pitch? The providers.

2024–2026: all three major vendors shipped **prefix caching** — exact, token-level, zero-risk — at roughly **90% discounts** on cached input, often **automatic**. The repeated context (system prompts, schemas, documents) is now discounted at the source.

*GPTCache did not die of a rival cache. It died of a **price change** — the promise from the Prologue, kept.*

ACT IV · THE SURVIVING STAKE

What is left to save

OUTPUT TOKENS

— prefix-cached calls still
generate at full price, at 3–6×
the input rate; only a semantic
hit avoids them

THE WHOLE PIPELINE

— a hit skips embed, search,
rerank, assemble; no provider
discount touches these

LATENCY

— seconds become
milliseconds; in voice, that is
not a discount, it is

THE PRODUCT

ACT IV · THE FORMULA

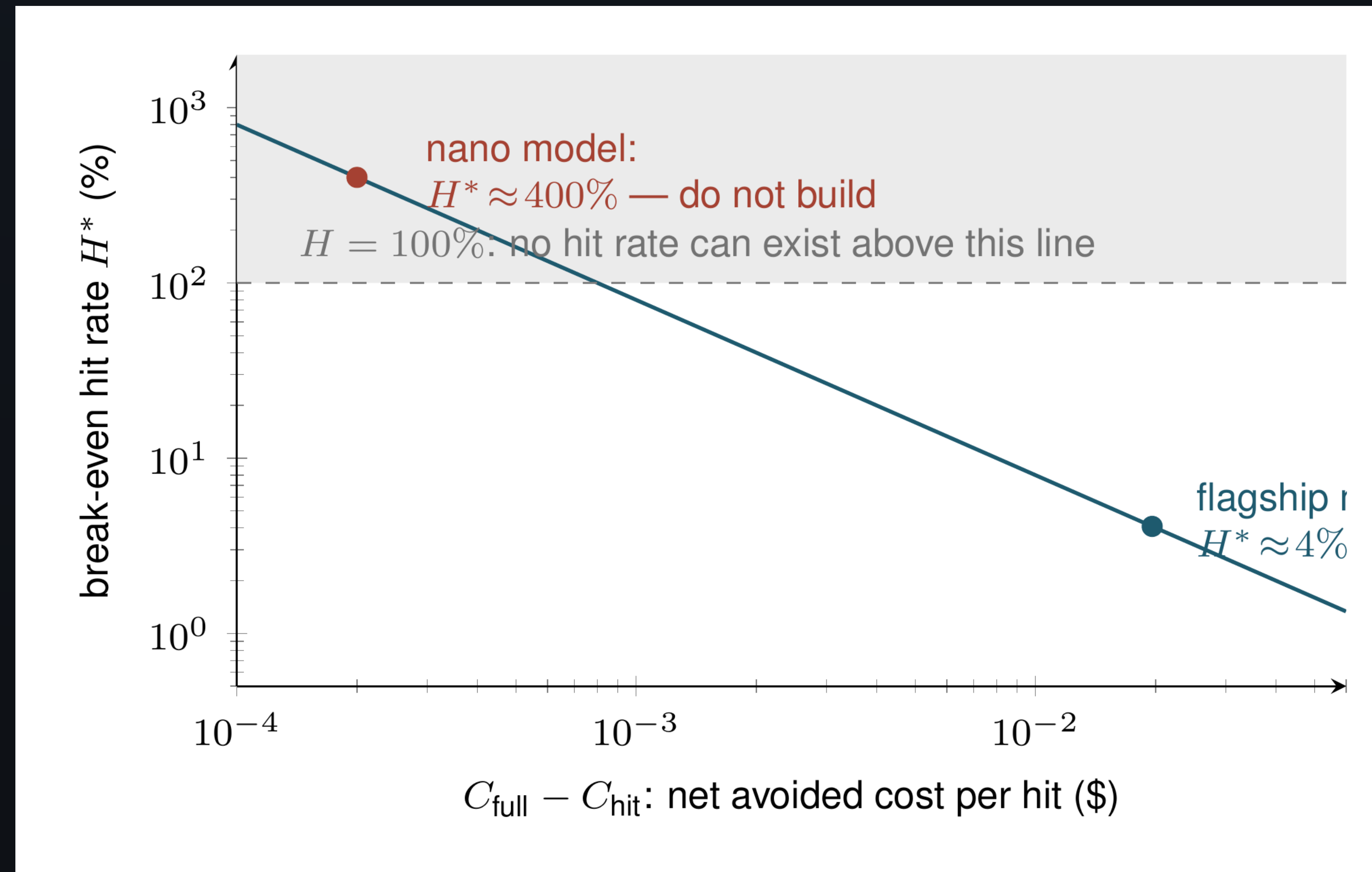
Break-even, with its teeth showing

$$H^* = \frac{C_{\text{ops}}}{N (C_{\text{full}} - C_{\text{hit}})}$$

The cache pays iff the hit rate clears H^* . C_{full} falls every quarter; C_{hit} is roughly constant physics.

Falling model prices raise the bar the cache must clear — the phase boundary migrates against you.

The phase boundary



*Above the $H = 100\%$ line no hit rate can exist — a whole region of configurations where the cache **cannot** pay. Read it like a phase diagram.*

ACT IV · TWO VERDICTS

Build. Do not build.

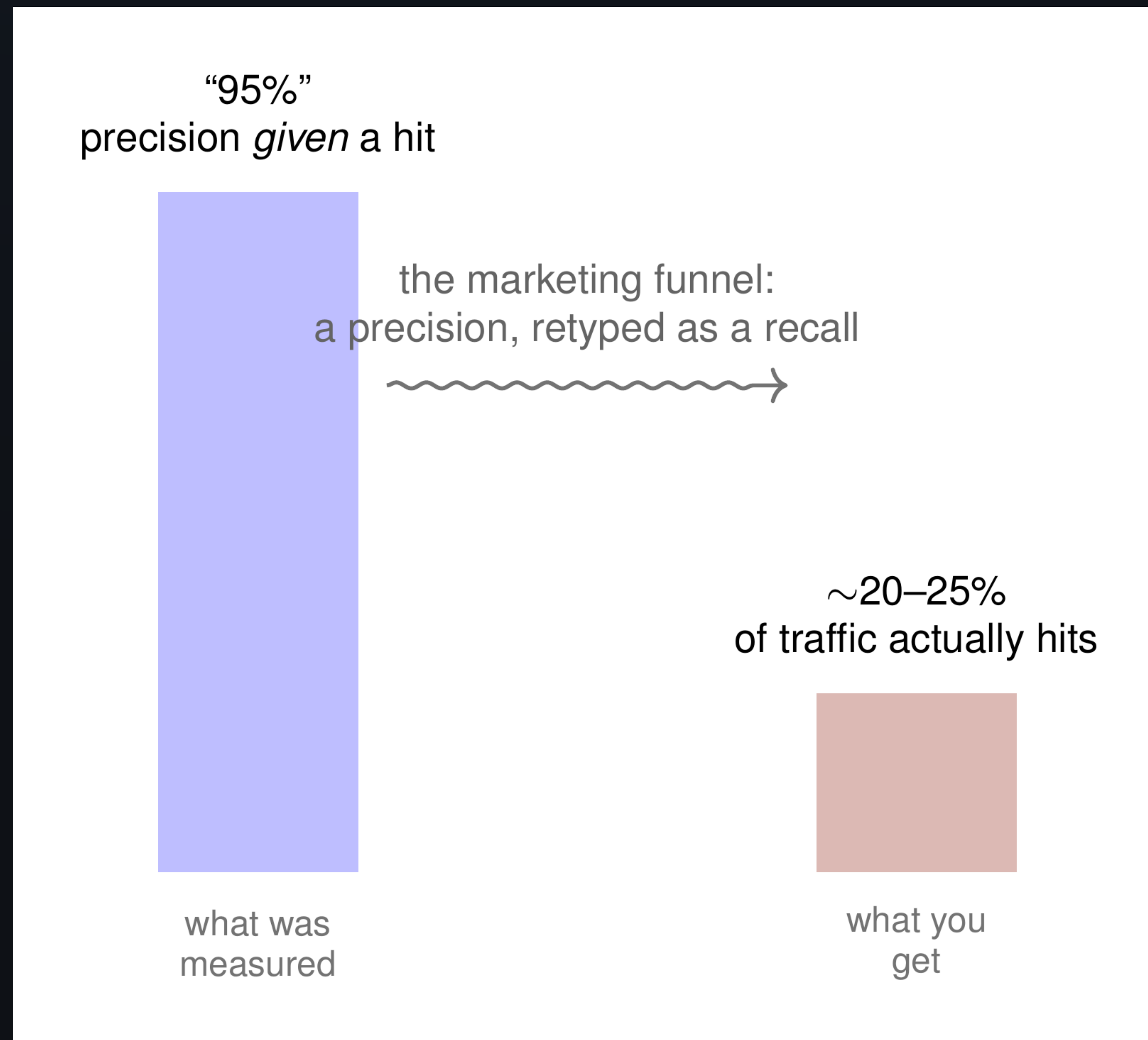
Flagship support bot: $N = 50,000 / \text{day}$,
 $C_{\text{full}} \approx \0.02 , $C_{\text{hit}} \approx \$0.0004$, $C_{\text{ops}} \approx \$40 \rightarrow$
 $H^* \approx 4\%$; observed FAQ hit rates run 40–60%.

Build.

Same bot on a nano-tier model: $C_{\text{full}} \approx \0.0006
 $\rightarrow H^* \approx 400\%$. **Unreachable at any hit rate.**

*“Do not build” is a **success** of the analysis, not a failure of nerve — the taught viability gate, now with a price tag.*

Accuracy–frequency laundering



Quiz 5 — rescue the cache

The nano-tier verdict came back
 $H^* \approx 400\%$. The team refuses to give
up. Which move rescues the
deployment?

(a) double the cache size (b) fine-
tune a better matcher to raise the hit
rate (c) lower τ to convert more
lookups into hits

Choose — then *say it back*: what does
your chosen knob actually do to H^* ?

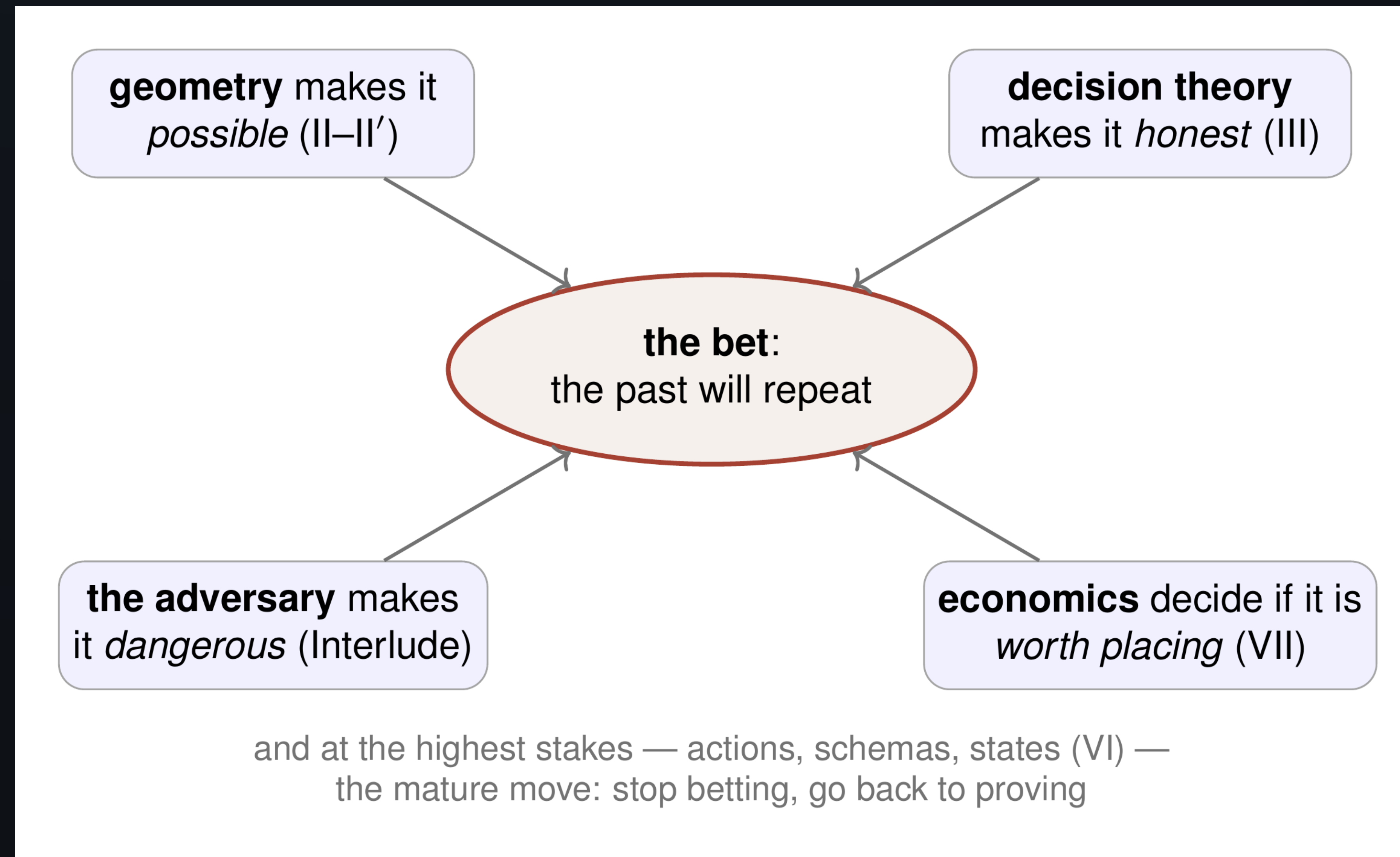
Quiz 5 — answered: none of them

H^* has no knob among the options:

- (a) size doesn't appear in the formula;
- (b) raises H , but no hit rate exceeds **100%**, and the bar is at 400%;
- (c) raises H and the error rate — you pay L more often to chase an unreachable bar.

The trap, again — and the lesson with it: when the bar itself is unreachable, every knob is the wrong knob. The only honest moves are "do not build" or a different stake: latency, where Δ is measured in conversions.

Four verdicts, one placement map



*Build; don't; verify first; or **stop betting and go back to proving** — where you sit on this map **is** your architecture.*



MILESTONE · ACT V OF V

The Wild

the 2026 market, read as
evidence — and the pattern
language

Read the market like a field naturalist

Believers: Redis (LangCache → Iris, still in preview), Azure APIM (ships it **with a warning label**), Kong & Portkey (enterprise-tier, candid ~20% hit rates). **Skeptics:** Cloudflare and Helicone — exact-match only, by design. **The pivot:** Memento, to KV offload. **The grave:** GPTCache.

*Two infra companies with the most caching DNA on earth **declined to ship it**. Every stance is evidence; the abstentions are the loudest.*

The tower of thresholds

Portkey

similarity, 0.95
higher = stricter →

Azure

distance, 0.05
← higher = looser

Kong

distance, 0.2
← higher = looser

AWS ref. arch.

similarity, 0.75
higher = stricter →

copy “0.2” from Kong into Portkey and you have not loosened the cache —
you have *inverted its meaning*

No agreement on what the dial means: Portkey similarity 0.95 (higher = stricter); Azure distance 0.05 (higher = looser); Kong distance 0.2; AWS similarity 0.75. Same number, opposite meanings.

Quiz 6 — the transplanted threshold

An engineer copies a battle-tested
`threshold: 0.2` from a Kong config
into a Portkey deployment, "to keep
the cache behavior identical."

***Predict** what the cache now does.*

*Then **say it back**: what two questions
must you ask of **any** product's threshold
before touching it?*

Quiz 6 — answered

Kong's 0.2 is a **distance** (strict);
Portkey's 0.2 is a **similarity** — near
the floor. The setting didn't loosen the
cache; it **inverted its meaning**: nearly
everything the index returns is now
served.

*Say-back: similarity or distance? which
direction is stricter? — check the sign of
the dial. And behind the incoherence, Act
II's verdict stands: every one of these dials
is rung one of the ladder.*

Ten patterns, one per whiteboard

Front Door · exact $\rightarrow$ semantic $\rightarrow$ pipeline, write-back · Prologue

Canonical Key · cache the meaning, not the wording · Prologue, Act III

Two-Tier Threshold · serve above, hint below · taught

Error Contract · budget δ , per-entry curves · Act II

Verified Hit · judge converts evidence $\rightarrow$ assurance · Act II

...and five more

Scoped Cache · scope key inside the key · [Interlude](#)

Version Pin · entries die with their geometry · [Act III](#)

Evidence Gauntlet · freshness vs the world, not the clock · [Act III](#)

Plan Cache · reuse the shape of work · [Act III](#)

Near-Miss Mine · the band as training data · [Act III](#)

Every anti-pattern is an operation without a policy

- **Global Threshold** — a serve operation without a decision policy
- **Unscoped Shared Cache** — a share operation without an access policy
- **Hit-Rate Worship** — a measurement operation without a truth policy
- **Checkbox Cache** — a deployment operation without an ownership policy

Hit rate is the dangerous metric: it rises as answers get worse. Watch the error, or the hit rate will watch you.

Reconstruct the map — the room speaks first

Five instruments crossed this stage today. **Before the reveal**: which failure does each one catch?

- the cap bound $e^{-d\varepsilon^2/2}$ · the error contract δ · the evidence gauntlet · the scoped key · the break-even H^*

And the five failures, shuffled: the money pit · the chance hit · the stale serve · the tenant leak · the confident wrong answer. Match them — out loud, in your own words.

ACT V · SAY IT BACK

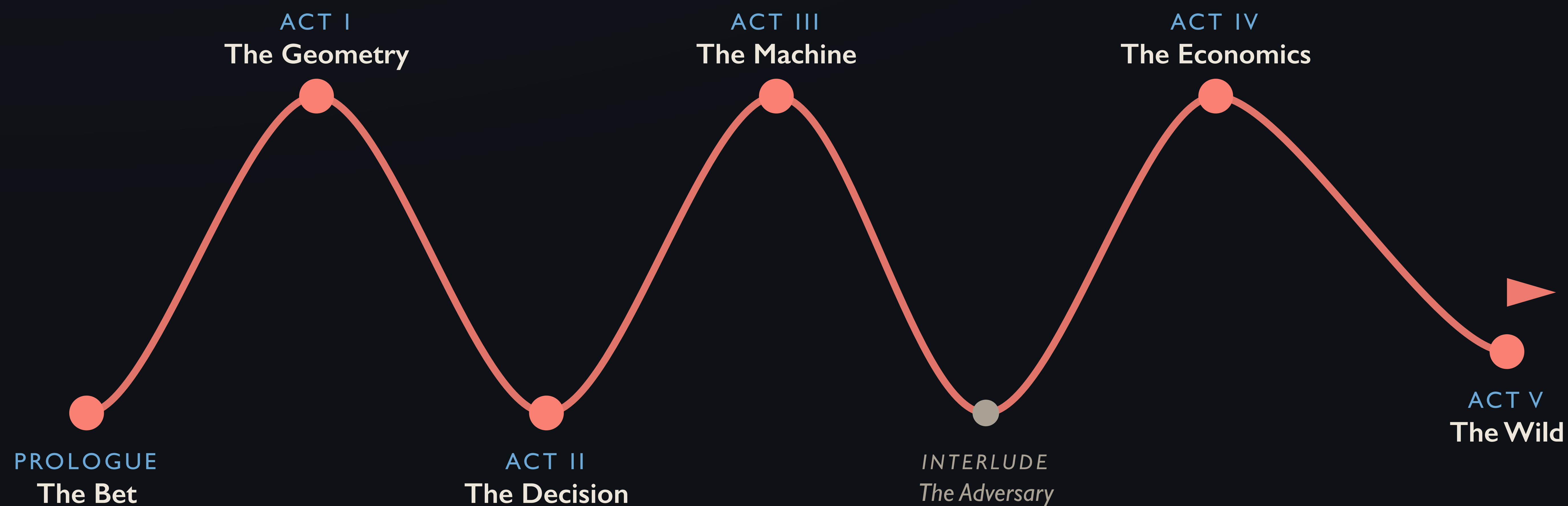
The map, revealed

- cap bound → **the chance hit** (there are none; hits are caused)
- error contract → **the confident wrong answer**
- evidence gauntlet → **the stale serve**
- scoped key → **the tenant leak**
- break-even H^* → **the money pit**

*Geometry makes the bet possible;
policy makes it honest; scope makes
it safe; arithmetic decides whether
to place it at all.*

PROMISES KEPT

The journey, traveled



The WHY

Repetition in real traffic is head-heavy and approximate; models answer the same question a thousand times in a thousand wordings. The semantic cache is the only organ in the whole architecture whose success is measured in **forward passes that never happen.**

A cache is a bet that the past will repeat. The craft is knowing — per entry, per user, per moment — when that bet is safe to place.

The WHAT

Five instruments: the **cap bound** (hits are caused), the **error contract** δ (a promise, not a hope), **VoS** with $p^* = (L + \kappa)/(g + L)$ (stakes set the bar), the **evidence gauntlet** (freshness vs the world), and H^* (the spreadsheet decides).

Geometry → possible. Policy → honest. Scope → safe. Arithmetic → whether at all.

PROMISES KEPT · 3 OF 3

The HOW

Monday morning, in order: turn on **prefix caching** (free, riskless, ~60% of the win); run the **viability gate** (one in ten sampled queries matching at 0.85 — or do not build); deploy the **scalpel** only on dense, scoped slices; sign the **error contract**; grade samples **weekly**.

Prefix caching is the default. The semantic cache is a scalpel — and the spreadsheet, not the enthusiasm, decides where it cuts.

THE STONES IN THE RIVER · SUPPORTVECTORS

Readings, dependency-ordered

1. Wang & Isola — alignment and uniformity on the hypersphere
2. vCache — verified caching and the error contract
3. GroundedCache — evidence-grounded freshness
4. The adversarial-hubness and cache-attack literature
5. **CacheCraft** — the full monograph, bears included

Read them in this order: each one is the answer to a question the previous one leaves open.

Thank you

The cheapest retrieval is the one that never runs. The costliest answer is the stale one served with confidence. Craft is knowing which sentence is speaking.